# Sentinel: A Rust-Only Signature-Based Virus Scanner

513559004 Jsaon Chia-Sheng Lin; 313264012 Ching-Chung Chen

2026-04-22

**Abstract**

Sentinel is a safe, read-only, Rust-based educational virus scanner for Project I in Network Security Practice. The scanner accepts an explicit target path, loads a structured JSON signature database, computes MD5 and SHA-256 digests incrementally, uses a Bloom filter as a negative pre-check before exact hash-map verification, searches byte signatures with a streamed Aho-Corasick automaton, skips symbolic links by default, separates heuristic warnings from confirmed signature matches, and writes JSON plus Markdown reports. The heuristic layer implements three bounded static checks: process-injection API names, executable-like magic bytes inside text-like files, and high byte entropy. This report is the canonical Project I report for the Rust submission path. It adds the Rust execution template and documents the algorithm and implementation excerpts in an academic-paper style. The demo uses the official EICAR safe anti-malware test file generated by Rust at demo time, plus benign local fixtures. No live malware is used.

## Keywords

Rust, virus scanner, signature matching, Bloom filter, Aho-Corasick, streaming scan, safe malware demo, Network Security Practice.

## 1 Submission Scope

This document is the canonical report for Project I - Virus Scanner. The report source and compiled output are:

- `report/final-report.tex`
- `report/final-report.pdf`

The recorded team members are 513559004 Jsaon Chia-Sheng Lin and 313264012 Ching-Chung Chen. The implementation is designed for a controlled course demo, not for production antivirus deployment. It does not execute scanned files, delete files, quarantine files, upload files, follow symbolic links by default, or scan the whole machine without an explicit target.

## 2 Problem Statement

The official Project I task is to construct a virus scanner that can identify a known threat inside a directory, compare files against a signature database, and produce a security report. The central engineering problem is not simply to print a finding; it is to build a reproducible, explainable pipeline where each detected result can be traced back to the signature database, the matching method, and the scan metadata.

The Rust version of Sentinel treats the project as a static-analysis scanner: files are opened as bytes, read in bounded chunks, compared against configured hash and byte-pattern signatures, and summarized without mutating the target tree. Confirmed signature evidence is reported as `infected`. Weak indicator-only evidence is reported as `suspicious`.

## 3 Rust-Only Architecture

| Rust module | Responsibility |
|---|---|
| `rust/src/main.rs` | CLI routing, default paths, version command, signature validation command, scan command, exit-code policy. |
| `rust/src/signatures.rs` | JSON signature loading, schema validation, normalization, hash index construction, pattern matcher extraction. |
| `rust/src/bloom.rs` | Bloom-filter set for `algorithm:digest` membership prechecks. |
| `rust/src/patterns.rs` | Aho-Corasick trie, failure-link construction, streamed byte-pattern scanning, pattern-offset reporting. |
| `rust/src/scanner.rs` | Directory traversal, symlink policy, chunked file reads, MD5/SHA-256 hashing, heuristic sampling, result classification, report assembly. |
| `rust/src/reporting.rs` | JSON and Markdown report serialization. |

## 4 Signature Database

The Rust scanner uses `signatures/malware-signatures.json`. The current course-demo signature is the official EICAR safe anti-malware test file, with three matchers:

| Matcher | Purpose in the Rust scanner |
|---|---|
| `md5` | Exact-file detection for the generated EICAR demo fixture. |
| `sha256` | Stronger exact-file detection for the same EICAR fixture. |
| `hex_pattern` | Byte-pattern detection for the bitwise comparison requirement. |

The Rust loader validates the signature schema and builds two search structures: a hash index for exact verification and a pattern list for automaton construction. This keeps signature loading separate from scanning, which makes the runtime path easier to test and explain.

## 5 Data Structures

| Structure | Used for | Reason |
|---|---|---|
| `Vec<Signature>` | Validated signature records | Keeps the database stable during one scan run. |
| `nested HashMap index` | Exact hash lookup | Gives direct lookup by algorithm and digest after the Bloom precheck; the concrete Rust type is a map from algorithm to digest to matching signature entries. |
| `HashBloomFilter` | Negative pre-check for hashes | Quickly rejects digests that cannot match while never accepting a positive without exact map verification. |
| `PatternEngine` | Shared Aho-Corasick automaton | Compiles all byte patterns once per scan. |

| | | |
|---|---|---|
| `PatternStreamScanner` | Per-file automaton state | Preserves pattern state across chunk boundaries. |
| `Vec<HeuristicFinding>` | Static warning evidence | Keeps weak indicators separate from confirmed signature matches. |
| `Vec<FileResult>` | Report rows | Preserves all clean, infected, suspicious, skipped, and error results for JSON/Markdown output. |

# 6 Rust Heuristic Analysis

The official brief asks for a basic heuristic ruleset to flag suspicious file behavior. Sentinel keeps this layer deliberately bounded and non-destructive: heuristics are computed from the same in-memory byte sample already read during static scanning. A heuristic finding cannot override the signature result model; without an exact hash or byte-pattern signature, it can only produce `suspicious`.

| Rule | Rust condition | Reason |
|---|---|---|
| `api-name-indicator` | The bounded text view contains process-injection API names such as `CreateRemoteThread`, `VirtualAllocEx`, or `WriteProcessMemory`. | Course-aligned example of suspicious behavior without claiming confirmed malware. |
| `magic-mismatch` | A text-like file begins with executable magic bytes such as `MZ` or `ELF`. | Flags a common hiding pattern: content type does not match the visible extension. |
| `high-entropy-sample` | A sample of at least 256 bytes reaches `7.5` bits per byte or higher. | Flags packed, encrypted, or compressed content as weak evidence for review. |

The Rust report records this layer as `heuristic_engine=bounded-static-byte-sample` and lists the active rule identifiers in `heuristic_rules`. This makes the generated report auditable: a reviewer can see not only the result, but which static rule family was enabled during the scan.

# 7 Core Algorithm

## 7.1 Algorithm 1: Rust Scan Pipeline

```
Input:
  target_path T
  signature_database_path S
  json_report_path J
  markdown_report_path M

Output:
  JSON report J
  Markdown report M
  process exit code 1 if confirmed infections exist, else 0
```

```
1. D <- load_signature_database(S)
2. P <- PatternEngine::new(D.patterns)
3. H <- D.hash_filter
4. R <- empty result list
5. For each path p collected from T in deterministic sorted order:
     a. If p is a symbolic link:
          append skipped result to R
          continue
     b. Open p as bytes
     c. Initialize MD5 context, SHA-256 context, and P.stream()
     d. While a chunk is available:
          update MD5 with chunk
          update SHA-256 with chunk
          feed chunk into the Aho-Corasick stream
          append chunk bytes to bounded heuristic sample if sample is not full
     e. For each computed digest d:
          if H.might_contain(algorithm:d) is false:
              skip exact lookup for that digest
          else:
              verify against D.hash_index[algorithm][d]
     f. Add byte-pattern matches returned by the stream scanner
     g. Run heuristic rules over the bounded sample
    h. Classify:
          if any exact signature or byte-pattern match -> infected
          else if any heuristic finding -> suspicious
          else -> clean
     i. append FileResult to R
6. Build summary and findings from R
7. Write J and M
8. Return exit code 1 if summary.infected > 0, else 0
```

## 7.2 Algorithm 2: Streamed Aho-Corasick Matching

```
Build phase:
1. Insert every configured byte pattern into a trie.
2. For each terminal trie node, store the corresponding pattern index.
3. Traverse the trie breadth-first.
4. For each node and transition byte:
     a. Follow failure links until a fallback transition exists or root is hit.
     b. Store the fallback state as the child node's failure link.
     c. Inherit output pattern indexes from the failure state.

Scan phase:
1. Start with state = root and absolute offset = 0.
2. For each byte b in each file chunk:
     a. Follow failure links until state has transition b or root is reached.
     b. Move to the next state for b, or root if absent.
     c. For each output pattern at the current state:
          pattern_offset = absolute_position + 1 - pattern_length
          record signature id, matcher type, pattern length, and offset
3. Preserve state between chunks so split signatures still match.
```

# 8 Rust Execution Template

The following template is the Rust run path used for this report. It requires Cargo and reproduces the scanner verification and demo evidence.

```
cd projects/project-i-virus-scanner/rust

cargo fmt --check
cargo test
cargo clippy --all-targets -- -D warnings

cargo run --quiet -- validate-signatures \
  --signatures ../signatures/malware-signatures.json

cargo run --quiet -- prepare-eicar-demo \
  --target ../demo/demo-tree

cargo run --quiet -- verify-demo \
  --target ../demo/demo-tree \
  --signatures ../signatures/malware-signatures.json

cargo run --quiet -- scan \
  --target ../demo/demo-tree \
  --signatures ../signatures/malware-signatures.json \
  --json ../reports/demo-report.json \
  --markdown ../reports/demo-report.md

cargo run --quiet -- write-evidence \
  --target ../demo/demo-tree \
  --signatures ../signatures/malware-signatures.json \
  --report ../reports/demo-report.json \
  --report ../reports/demo-report.md \
  --output ../reports/demo-evidence-manifest.json

# Expected scan summary:
# Sentinel v2 Rust scan complete: scanned=5 infected=1 suspicious=1 clean=3 errors=0
#
# Expected exit code:
# 1, because the generated EICAR safe test file is intentionally detected.
```

# 9 Implementation Excerpts

This section shows compact excerpts from the actual Rust implementation. The snippets are included to make the report reviewable like an academic submission: the algorithm can be read from the paper, and the complete source can be checked in the repository.

## 9.1 CLI Template and Exit Policy

```
fn run(args: Vec<String>) -> Result<i32, String> {
    match args.first().map(String::as_str) {
        Some("--version") | Some("-V") => {
            println!("sentinel-v2-rust {VERSION}");
            Ok(0)
        }
        Some("validate-signatures") => {
            let signatures = parse_value_arg(&args[1..], "--signatures")
```

```
                .map(PathBuf::from)
                .unwrap_or_else(default_signature_path);
            let database = load_signature_database(&signatures)?;
            println!(
                "signature database ok: schema={} signatures={} hash_matchers={}
    patterns={}",
                database.schema_version,
                database.signatures.len(),
                database.hash_matcher_count(),
                database.patterns.len()
            );
            Ok(0)
        }
        Some("scan") => {
            let scan_args = parse_scan_args(&args[1..])?;
            let database = load_signature_database(&scan_args.signatures)?;
            let report = scan_directory(&scan_args.target, &database)?;
            write_json_report(&report, &scan_args.json_report)?;
            write_markdown_report(&report, &scan_args.markdown_report)?;

            if report.summary.infected > 0 {
                Ok(1)
            } else {
                Ok(0)
            }
        }
        _ => {
            print_usage();
            Ok(2)
        }
    }
}
```

## 9.2 Chunked File Scan

```
fn scan_file_content(
    path: &Path,
    pattern_engine: &PatternEngine,
) -> Result<ScannedFileContent, String> {
    let mut file = File::open(path).map_err(|err| err.to_string())?;
    let mut md5_context = md5::Context::new();
    let mut sha256 = Sha256::new();
    let mut pattern_stream = pattern_engine.stream();
    let mut heuristic_sample = Vec::new();
    let mut size_bytes = 0_u64;
    let mut buffer = vec![0_u8; CHUNK_SIZE_BYTES];

    loop {
        let bytes_read = file.read(&mut buffer).map_err(|err| err.to_string())?;
        if bytes_read == 0 {
            break;
        }

        let chunk = &buffer[..bytes_read];
        md5_context.consume(chunk);
        sha256.update(chunk);
        pattern_stream.feed(chunk);
```

```
        size_bytes += bytes_read as u64;

        if heuristic_sample.len() < HEURISTIC_SAMPLE_LIMIT_BYTES {
            let remaining = HEURISTIC_SAMPLE_LIMIT_BYTES - heuristic_sample.len();
            heuristic_sample.extend_from_slice(&chunk[..chunk.len().min(remaining)]);
        }
    }

    Ok(ScannedFileContent {
        size_bytes,
        md5: format!("{:x}", md5_context.compute()),
        sha256: format!("{:x}", sha256.finalize()),
        pattern_matches: pattern_stream.finish(),
        heuristic_sample,
    })
}
```

### 9.3 Bloom-Then-Exact Hash Verification

```
fn find_hash_matches(
    hashes: &HashMap<String, String>,
    database: &SignatureDatabase,
) -> Vec<HashSignatureMatcher> {
    let mut matches = Vec::new();
    for (algorithm, digest) in hashes {
        if !database.hash_filter.might_contain(algorithm, digest) {
            continue;
        }
        if let Some(digest_map) = database.hash_index.get(algorithm) {
            if let Some(found) = digest_map.get(digest) {
                matches.extend(found.iter().cloned());
            }
        }
    }
    matches
}
```

### 9.4 Streamed Pattern Matching

```
pub fn feed(&mut self, data: &[u8]) {
    for (chunk_position, byte) in data.iter().copied().enumerate() {
        self.state = self.engine.transition(self.state, byte);
        let absolute_position = self.offset + chunk_position;

        for pattern_index in self.engine.nodes[self.state].outputs.iter().copied() {
            if self.seen.insert(pattern_index) {
                let pattern = &self.engine.patterns[pattern_index];
                let offset = absolute_position + 1 - pattern.pattern.len();
                self.matches.push(PatternMatch {
                    signature: pattern.signature.clone(),
                    matcher: pattern.matcher_type.clone(),
                    pattern_offset: offset,
                    pattern_bytes: pattern.pattern.len(),
                });
            }
```

```
        }
    }

    self.offset += data.len();
}
```

## 9.5 Bounded Heuristic Rules

```
fn run_heuristics(path: &Path, content: &[u8]) -> Vec<HeuristicFinding> {
    let sample_len = content.len().min(1024 * 1024);
    let text = String::from_utf8_lossy(&content[..sample_len]);
    let mut findings = Vec::new();

    let indicators: Vec<&str> = ["CreateRemoteThread", "VirtualAllocEx", "
    WriteProcessMemory"]
        .into_iter()
        .filter(|indicator| text.contains(indicator))
        .collect();

    if !indicators.is_empty() {
        findings.push(HeuristicFinding {
            rule_id: "api-name-indicator".to_string(),
            severity: "medium".to_string(),
            description: "Suspicious API-name strings appear in the file.".to_string()
    ,
            evidence: indicators.join(", "),
        });
    }

    if starts_with_executable_magic(content) && has_text_like_extension(path) {
        findings.push(HeuristicFinding {
            rule_id: "magic-mismatch".to_string(),
            severity: "high".to_string(),
            description: "Executable-like magic bytes appear in a text-like file.".
    to_string(),
            evidence: format!(
                "extension={} magic=MZ",
                path.extension()
                    .and_then(|value| value.to_str())
                    .unwrap_or("<none>")
            ),
        });
    }

    if content.len() >= HIGH_ENTROPY_MIN_BYTES {
        let entropy = shannon_entropy(content);
        if entropy >= HIGH_ENTROPY_THRESHOLD {
            findings.push(HeuristicFinding {
                rule_id: "high-entropy-sample".to_string(),
                severity: "low".to_string(),
                description: "The bounded sample has high byte entropy.".to_string(),
                evidence: format!("entropy={entropy:.2} bits/byte sample_bytes={}",
    content.len()),
            });
        }
    }
```

```
    findings
}
```

## 10 Result Classification

| Status | Rust condition |
|---|---|
| `infected` | At least one exact MD5/SHA-256 signature match or one configured byte-pattern match is found. |
| `suspicious` | No signature match exists, but a heuristic indicator is present in the bounded sample. |
| `clean` | No signature match and no heuristic finding exists. |
| `skipped` | The path is a symbolic link and the default symlink policy is `skip`. |
| `error` | The scanner cannot read metadata or file content for the path. |

This ordering prevents heuristic-only evidence from being overstated as a confirmed infection. It also makes the expected course demo easy to audit: the generated EICAR safe test file is confirmed by MD5, SHA-256, and hex-pattern evidence, while the API-name fixture remains only suspicious.

## 11 Rust Evaluation

The Rust demo command produced the following summary:

| Metric | Result |
|---|---|
| Files scanned | 5 |
| Infected | 1 |
| Suspicious | 1 |
| Clean | 3 |
| Skipped | 0 |
| Errors | 0 |

The Rust report metadata records:

- `hash_filter`: `bloom-filter`
- `hash_filter_policy`: `precheck-then-exact-hash-map`
- `pattern_engine`: `aho-corasick-byte-automaton`
- `pattern_count`: `1`
- `automaton_states`: `69`
- `chunk_size_bytes`: `1048576`
- `heuristic_engine`: `bounded-static-byte-sample`
- `heuristic_rules`: `api-name-indicator`, `magic-mismatch`, `high-entropy-sample`
- `symlink_policy`: `skip`

The Rust test suite now contains 13 tests, including dedicated heuristic tests for process-injection API names, text-file magic mismatch detection, high entropy detection, repeated-byte entropy control, and Rust evidence-manifest generation. The Rust verification gate passed locally with formatting, tests, Clippy, signature validation, demo summary verification, and evidence-manifest generation:

```
cargo fmt --check
cargo test
cargo clippy --all-targets -- -D warnings
cargo run -- prepare-eicar-demo --target ../demo/demo-tree
cargo run -- verify-demo \
  --target ../demo/demo-tree \
  --signatures ../signatures/malware-signatures.json
cargo run -- write-evidence \
  --target ../demo/demo-tree \
  --signatures ../signatures/malware-signatures.json \
  --report ../reports/demo-report.json \
  --report ../reports/demo-report.md \
  --output ../reports/demo-evidence-manifest.json

Rust demo verification passed: scanned=5 infected=1 suspicious=1 clean=3 skipped=0
    errors=0
```

## 12 Security Boundary and Ethics

The scanner is intentionally conservative. The current demo uses the official EICAR safe anti-malware test file because the project brief asks for EICAR, but the literal `eicar.com.txt` file is generated at demo time by Rust and is not committed or exported. EICAR is not live malware. The scanner does not execute scanned files, detonate samples, mutate targets, quarantine files, or upload telemetry. MD5 is used only as a legacy signature-matching demonstration and is paired with SHA-256 and byte-pattern evidence; it is not presented as a modern cryptographic integrity guarantee.

## 13 Limitations

Sentinel is not production antivirus software. Its coverage is limited to the configured signatures and simple heuristic indicators. It does not implement archive unpacking, PE parsing, sandbox execution, cloud reputation, YARA rules, or real-time file-system monitoring. These omissions are intentional for the course setting: the goal is a clear, reproducible, safe scanner whose algorithm and outputs can be reviewed directly.

## 14 Conclusion

The Rust Sentinel implementation satisfies the Project I educational goal: it scans an explicit directory, validates a structured signature database, combines Bloom-filter pre-checking with exact hash verification, performs stream-safe Aho-Corasick byte-pattern detection, reports weak heuristics separately from confirmed signatures, and generates reproducible JSON, Markdown, and evidence-manifest artifacts. This final Rust report documents both the algorithm and the actual Rust template used to build, test, validate, and run the scanner.

## 15 Rust Screenshot Evidence

This appendix records the Rust evidence screenshots generated for the report.

```
Sentinel v2 Rust command surface

$ cargo run -- validate-signatures --signatures ../signatures/malware-signatures.json
signature database ok: schema=1.0 signatures=1 hash_matchers=2 patterns=1

$ cargo run -- prepare-eicar-demo --target ../demo/demo-tree
EICAR demo fixture written: ../demo/demo-tree/nested/level-1/level-2/eicar.com.txt bytes=68

$ cargo run -- verify-demo --target ../demo/demo-tree --signatures
../signatures/malware-signatures.json
Rust demo verification passed: scanned=5 infected=1 suspicious=1 clean=3 skipped=0 errors=0
```

Figure 1: Rust command-surface evidence showing the local Cargo/Rust toolchain, version output, CLI usage, and safe execution boundary.

```
Signature database: signatures/malware-signatures.json

Signature: sig-eicar-standard-antivirus-test-file
Name: EICAR Standard Anti-Virus Test File
Category: safe-standard-test-file
Severity: critical

Matchers:
- md5     44d88612fea8a8f36de82e1278abb02f
- sha256  275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f
- hex_pattern, 68 bytes, matched at offset 0

No live malware is used. The EICAR file is generated at demo time and ignored by
Git/export.
```

Figure 2: Rust signature-validation evidence showing the shared JSON database, hash matchers, and byte-pattern matcher accepted by the Rust loader.

```
$ make rust-evidence

EICAR demo fixture written: ../demo/demo-tree/nested/level-1/level-2/eicar.com.txt bytes=68
Sentinel v2 Rust scan complete: scanned=5 infected=1 suspicious=1 clean=3 errors=0
Rust evidence manifest written: ../reports/demo-evidence-manifest.json

Expected detection exit code: 1
Reason: generated EICAR safe test file was intentionally detected.
```

Figure 3: Rust demo-scan evidence showing the expected safe-fixture detection summary and the expected detection exit code.

```
reports/demo-report.md

Summary:
- Files scanned: 5
- Infected: 1
- Suspicious: 1
- Clean: 3
- Skipped: 0
- Errors: 0

Findings:
- nested/level-1/level-2/eicar.com.txt
  status: infected
  severity: critical
  evidence: md5, sha256, hex_pattern@0

- suspicious/api-names-fixture.txt
  status: suspicious
  evidence: api-name-indicator
```

Figure 4: Rust report evidence showing the generated Markdown report metadata, Bloom-filter pre-check, Aho-Corasick engine, summary, and finding rows.

```
Rust verification gate

$ cargo fmt --check
passed

$ cargo test
13 passed; 0 failed

$ cargo clippy --all-targets -- -D warnings
passed

$ cargo run -- verify-demo --target ../demo/demo-tree --signatures
../signatures/malware-signatures.json
Rust demo verification passed: scanned=5 infected=1 suspicious=1 clean=3 skipped=0 errors=0
```

Figure 5: Rust verification-gate evidence showing source/lockfile presence, formatting, tests, Clippy, signature validation, and demo parity passing.

```
Submission package decision

Project I requires a safe EICAR demo.

Implemented policy:
- Rust prepare-eicar-demo generates the exact 68-byte EICAR test file.
- Sentinel scans the nested demo tree and detects eicar.com.txt.
- The generated literal file is ignored by Git and excluded from private-repo export.
- No live malware is stored, executed, quarantined, uploaded, or mutated.

Remaining official submission item:
- Create the private GitHub/GitLab repository and record URL/commit hash.
```

Figure 6: Submission-package evidence showing the Rust checklist and Rust demo-report command recorded for final delivery.

## 16 References


- EICAR/CARO Anti-Malware Test File guidance, used as the required safe antivirus-test fixture for the controlled demo.
- NIST SP 800-83 Rev. 1, Guide to Malware Incident Prevention and Handling.
- OWASP File Upload Cheat Sheet, used as background for safe handling of untrusted files.
- Aho, A. V. and Corasick, M. J. Efficient string matching: an aid to bibliographic search. Communications of the ACM, 1975.